

Malicious URL Detection System

SYSTEM FLOW

Step 1 — You navigate to a page

Example: you type `https://paypal-security-alert.com/verify` and hit Enter.

Step 2 — background.js fires

Chrome triggers the `tabs.onUpdated` listener the moment the page finishes loading (`status === "complete"`). It grabs the URL from the tab.

Step 3 — POST /predict is called

`background.js` sends the URL to your Flask API:

`json`

POST `http://localhost:5000/predict`

```
{"url": "https://paypal-security-alert.com/verify"}
```

Step 4 — Flask API processes it

Two-layer check:

- **Layer 1** — Is it in the whitelist (54 trusted domains)?
 - Yes → instantly returns BENIGN, skip model
 - No → go to Layer 2
- **Layer 2** — Build 548-feature vector:
 - Extract 48 heuristic features
 - Scale with `StandardScaler`
 - Transform with char + word TF-IDF vectorizers
 - Feed into Random Forest → get malicious probability
 - Compare against threshold (0.44) → MALICIOUS or BENIGN

Step 5 — Badge updates instantly

`background.js` receives the result and updates the extension icon:

-  ! red badge → MALICIOUS
-  ✓ green badge → BENIGN
-  ? gray badge → API unreachable
- Result is also cached in `chrome.storage.session` by `tabId`.

Step 6 — Warning banner appears (if MALICIOUS)

background.js sends a message to content.js which injects a red warning strip at the top of the page:

⚠ Malicious URL Detected — This page may be dangerous. Confidence: 87.5%

Step 7 — You click the extension icon

popup.js opens and:

- Checks session cache first (no need to call API again)
- Renders the verdict card — MALICIOUS/BENIGN, confidence bar, URL

Step 8 — POST /explain is called

popup.js simultaneously calls:

json

POST http://localhost:5000/explain

```
{"url": "https://paypal-security-alert.com/verify", "num_features": 20}
```

LIME perturbs the feature vector ~5000 times, runs each through the Random Forest, fits a linear model and returns top features.

Step 9 — Explanation renders in popup

After filtering out char_* and word_* features, top 3 human-readable reasons appear:

-  Brand impersonation detected — detected
-  Fake brand domain pattern — detected
-  Suspicious domain ending (.tk .xyz) — detected

1. CREATE LIME BACKGROUND NPZ

```
python -c "  
import sys, os  
sys.path.insert(0, 'app')  
  
# Remove old one first (built with 548 features — now incompatible) → Don't Copy This Already You Don't  
Have lime_background.npz  
bg_path = 'models/lime_background.npz'  
if os.path.exists(bg_path):  
    os.remove(bg_path)  
    print('Removed old background')  
  
from model_loader import classifier  
print('Building new background (552 features)...')  
classifier._load_background()  
print('Done — models/lime_background.npz saved')  
"
```

2. REBUILD LIME BACKGROUND:

```
python -c "  
import sys, os  
sys.path.insert(0, 'app')  
os.remove('models/lime_background.npz')  
from model_loader import classifier  
classifier._load_background()  
print('Done')  
"
```

3. TO RUN THIS SYSTEM

```
cd "D:\Research Project\Research_Working_dir_2\malicious_url_detection"  
.\venv\Scripts\activate  
python app/app.py
```